

Android programmering, eksamen høst 2024

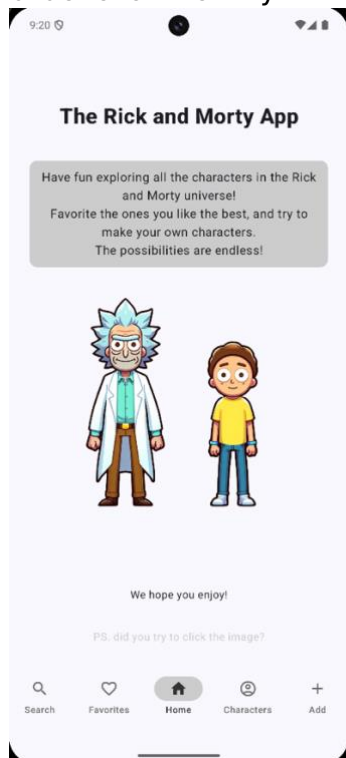
Tema: Rick and Morty

Funksjonsnavn	Beskrivelse
initializeDatabase	Databasen blir bygget og initialisert med Room ved å opprette en instanse av «AppDatabase», og dette er det første som kjøres i koden.
getAllMyCharacters	Henter alle karakterene fra databasen vår via spørring fra DAO-filen.
insertMyCharacter	Registrerer en ny karakter i databasen vår som brukeren selv har opprettet via spørring fra DAO-filen.
deleteCharacterById	Sletter den karakteren fra databasen vår utifra hvilken id som matcher id den har, via en spørring fra DAO-filen.
characterExists	Sjekker om det allerede eksisterer en karakter i databasen vår med det navnet brukeren forsøker å opprette en ny karakter med, da vi kun ønsker å godta unike navn på karakterene.
getCharacterById	Henter tilsvarende karakter fra API ´et ut ifra hvilken id brukeren har skrevet inn.
getAPICharacters	Henter en liste med karakterene fra API ´et for å kunne vise karakterene uten at brukeren trenger å søke på noe. Returnerer eventuelt en tom liste dersom noe feiler under HTTP-kallet.
setSearchedCharacter	Henter en spesifikk karakter fra API ´et og oppdaterer state på den tomme listen vi først opprettet, før den kaller på get-metoden for å hente dataen om den karakteren og deretter oppdatere staten med karakteren som har tilsvarende id som brukeren har tastet inn.
setMyCharacters	Laster inn karakterene fra databasen vår inn i en tom liste ved å kalle på get-metoden, og gir den en state flow som kan oppdateres.
insertMyCharacter	Legger til nye karakterer i databasen vår og oppdaterer state på listen med karakterer.
deleteMyCharacter	Sletter en karakter fra databasen vår ved å kalle på spørringen fra DAO-filen, og oppdaterer state på listen med karakterer for å kunne fjerne karakteren fra både local storage og listen over brukerens karakterer i UI ´et.
getAPIIcons	Henter ut API bildene som en liste

getNextcharacterImage	Henter ut kun et API bilde om gangen og endrer til neste i lista når man trykker på bildet.
toggleFavorite	Når man trykker på hjerte ikonet så legger man til den karakteren som favoritt og blir deretter skrevet ut på Favorites siden

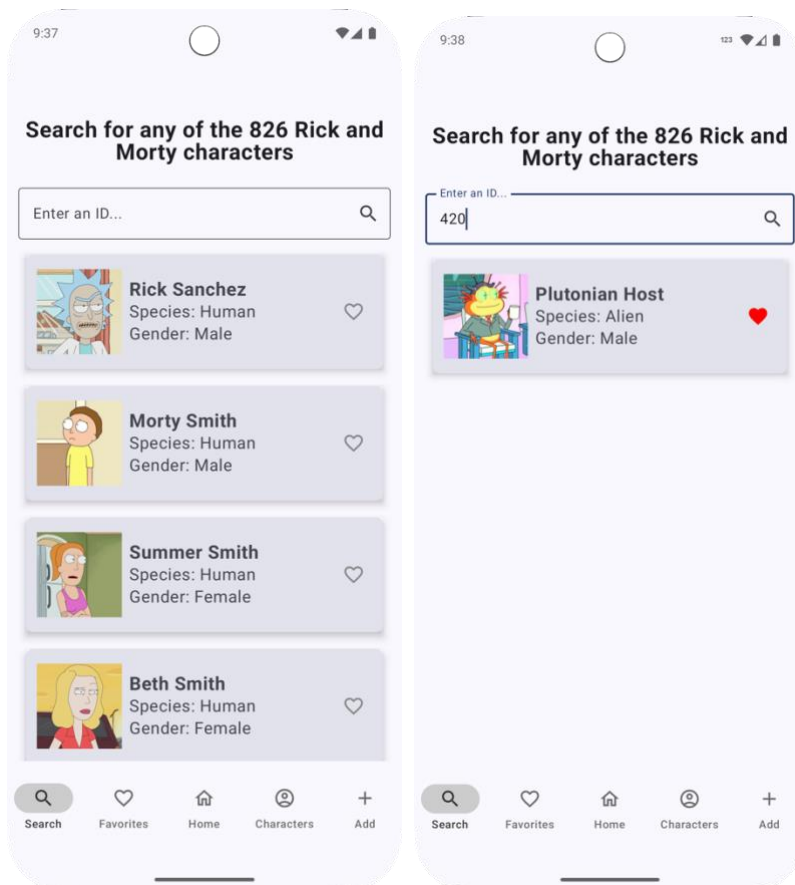
Skjerm 3 – Home

Vi valgte å ha en statisk skjerm ettersom ingen av de andre skjermene passet å ha som “homepage” som inneholder en velkomstmelding med hva appen har å tilby og et bilde, va la til slutt en funksjon som gjør at når man trykker på bildet ruller det mellom API bildene for hvert trykk.



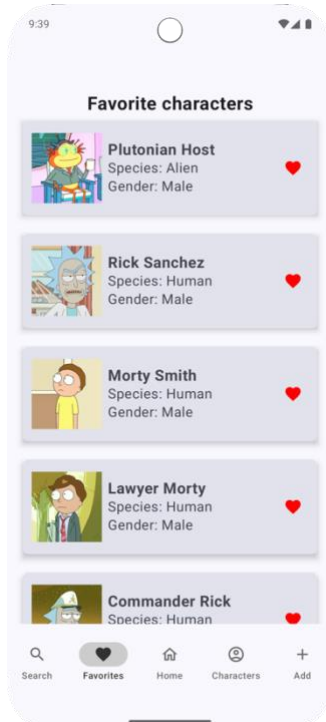
Skjerm 1 – Search

Her kan brukeren enten bla gjennom karakterene fra Rick and Morty API ´et, men kun de første 20 ettersom vi ikke ønsket å laste inn alle 826 karakterer på én gang, eller søke etter én karakter basert på id. Det er også mulig å trykke på hjertet ved siden av hver karakter for å legge de til som favoritt (disse vises da på skjerm 2).



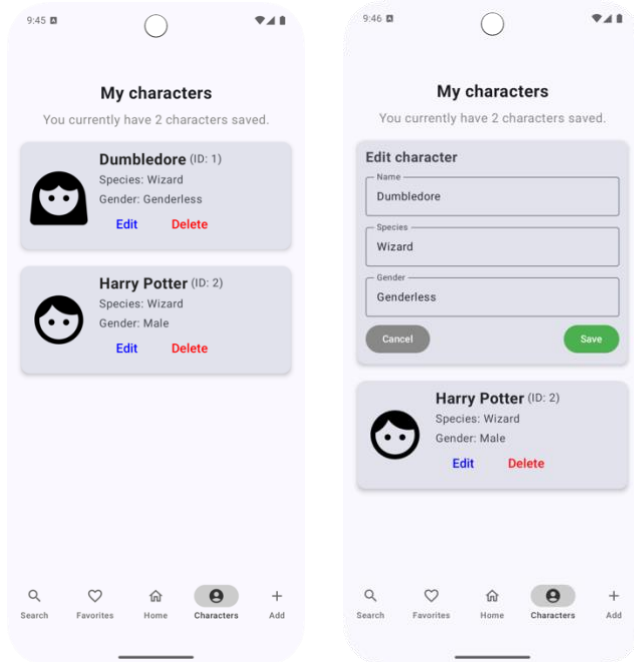
Skjerm 2 – Favorites

Som nevnt over så er det her alle karakterer havner etter man har trykket på hjertet på forrige skjerm, og det kan fjernes både fra denne og forrige skjerm dersom brukeren ikke ønsker å ha de som favoritt lenger.



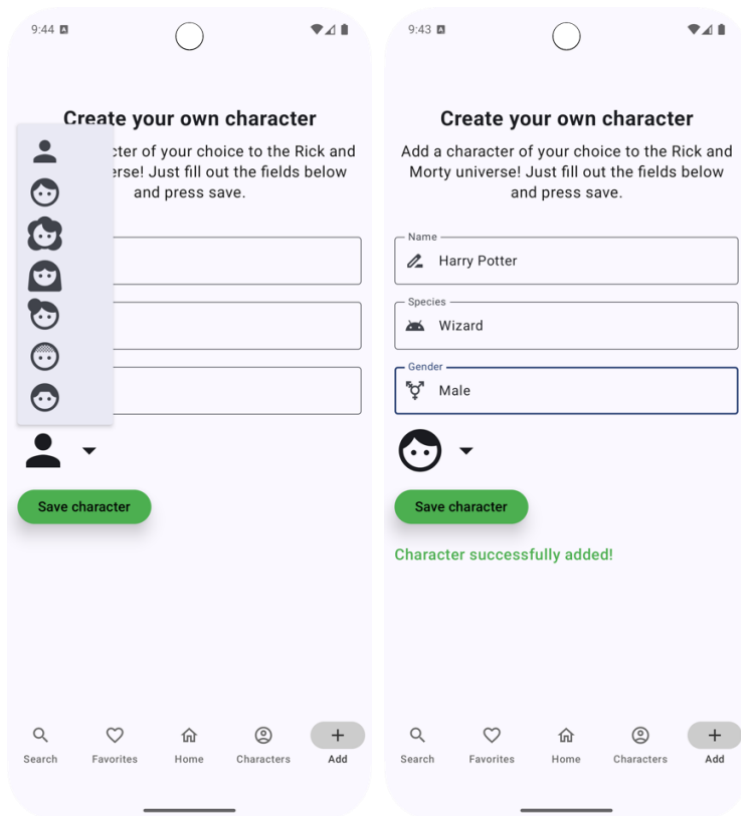
Skjerm 4 – Characters

Her vises alle karakterene som brukeren har opprettet inne på skjerm 5, og vi ønsket noe ala likt design som karakterene fra API ´et har. Det er mulig for brukeren å slette karakteren(e) de har opprettet, eller trykke på rediger-knappen og endre noe av dataen. Avataren kan ikke endres ettersom vi heller ønsker at brukeren lager en helt ny karakter dersom de ønsker å endre på alt.



Skjerm 5 – Add

På den siste skjermen har brukeren mulighet til å legge til sin egen karakter, og igjen valgte vi å ha de samme verdiene som vi hentet fra API ´et (name, species, gender). Det ble dog litt kjedelig å kun ha 3 tekstfelt for å definere en karakter, så vi la til muligheten for å kunne velge mellom et par avatarer som allerede ligger inne i appen - noe som kan utbedres med bildeopplastning direkte fra brukerens telefon etter hvert.



Hovedteknikker vi har brukt er blant annet ViewModel som håndterer data som vises på de forskjellige skjermene uten at man mister informasjon når man bytter mellom dem. Den gjør at man separerer logikk fra UI og gjør det mer oversiktlig og testbart.

```
@Composable
fun HomePageScreen(homePageViewModel: HomePageViewModel) {
    class HomePageViewModel : ViewModel() {

@Composable
fun AppNavigation(
    homePageViewModel: HomePageViewModel,
    addCharacterListViewModel: AddCharacterListViewModel,
    myCharacterListViewModel: MyCharacterListViewModel,
    apiCharacterViewModel: APICharacterViewModel,
    favoritedCharactersViewModel: FavoritedCharactersViewModel
) {
```

For å hente data fra API brukte vi Retrofit som er et HTTP-klientbibliotek som gjør det enkelt å sende og motta HTTP-forespørsler.

```
object APICharacterRepository {
    private val _okHttpClient = OkHttpClient.Builder()
        .addInterceptor(
            HttpLoggingInterceptor().setLevel(
                HttpLoggingInterceptor.Level.BODY
            )
        )
        .build()

    private val _retrofit = Retrofit.Builder()
        .client(_okHttpClient)
        .baseUrl("https://rickandmortyapi.com/api/")
        .addConverterFactory(
            GsonConverterFactory.create()
        )
        .build()

    private val _characterService = _retrofit.create(APICharacterService::class.java)
```

For å lagre data lokalt i appen når man lager sin egen karakter bruker vi Room som er et SQLite-abstraksjonsbibliotek som gjør at vi har en lokal database med informasjonen brukeren sender inn. Room håndterer alle operasjonene som feks create, read, update og delete.

```
@Dao
interface MyCharacterDao {
    @Query("SELECT * FROM MyCharacter")
    suspend fun getAllMyCharacters() : List<MyCharacter>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMyCharacter(myCharacter: MyCharacter) : Long

    @Query("SELECT * FROM MyCharacter WHERE id = :id")
    suspend fun getMyCharacterById(id: Int) : MyCharacter?

    @Query("SELECT * FROM MyCharacter WHERE name = :name LIMIT 1")
    suspend fun getCharacterByName(name: String): MyCharacter?

    @Update
    suspend fun updateMyCharacter(myCharacter: MyCharacter)

    @Query("DELETE FROM MyCharacter WHERE id = :characterId")
    suspend fun deleteMyCharacter(characterId: Int): Int
}
```

Med klassene våre har vi brukt PascalCase og flere av de har like type funksjoner som gjelder character og derfor lignende navn, men vi bruker tydelig ord foran som gjør det enkelt å skille de. Alle funksjoner som bruker API heter derfor APICharacter og alt som omhandler vår egen database heter MyCharacter.

Funksjonene våre har tydelig navn med akkurat hva den gjør, som get, update og deleteCharacter feks. Vi bruker også derfor camelCase for å gjøre det lett å lese og for å skille det fra hvordan vi skriver klassene.

For å samarbeide prøvde vi først codeTogether for å kode sammen samtidig, men siden ikke begge kunne bruke emulatoren for å sjekke appen ble det vanskelig. Det som funket best for oss var å heller bruke gitHub og fordele arbeidet så vi kan kode hver for oss. Det vi kunne gjort annerledes er å fordele arbeidet så man jobber med funksjoner som ligner på hverandre, bruker mye av de samme teknikkene eller kommuniserer med hverandre. Også at man møttes mer og jobbet med det fysisk sammen så man kan vise og forklare teknikkene man bruke til hverandre. Dette hadde gjort det enklere å få til andre funksjoner vi prøvde å få til, men som ble utfordrende så vi måtte endre planen vi opprinnelig hadde.